# Auditoria tecnica - Hackaton

25 de agosto de 2026 · Revision estatica, runtime local y UX

## Resumen ejecutivo

Hackaton tiene una base sana: Rust con SQL parametrizado, bcrypt con coste 13, autorizacion de administrador en servidor, aislamiento de sitios en iframe sandbox y build de frontend correcto. Antes de exponerlo fuera de desarrollo debe resolver secretos versionados, CORS permisivo, JWT persistente en el navegador y ausencia de defensa anti-fuerza-bruta. El login tiene personalidad fuerte en escritorio, pero falla visualmente en movil.

**Prioridad:** SEC-01, SEC-02, SEC-03, SEC-04 y SEC-05; despues PERF-01 y UX-01.

## Alcance y comprobaciones

Revision de Rust/Actix, React/TypeScript, CSS, Docker y configuracion; ejecucion local con PostgreSQL Docker; build, pruebas, auditoria npm e inspeccion Chromium a 929 px y 390 px. No es una prueba de penetracion ni una revision de infraestructura externa.

| Comprobacion | Resultado |
|---|---|
| **npm run build** | Pasa. JS 441.02 kB (135.78 kB gzip); CSS 121.91 kB (18.25 kB gzip). |
| **npm audit --omit=dev** | 0 vulnerabilidades conocidas en dependencias de produccion. |
| **cargo test -q** | Pasa. |
| **cargo fmt --check** | Falla: multiples modulos no cumplen el formato. |
| **cargo clippy -D warnings** | Falla: seis avisos tratados como error. |
| **cargo audit** | No disponible en esta maquina; advisories Rust sin verificar. |

## Seguridad

### SEC-01 · Alta · Secretos de desarrollo versionados

**Ubicacion:** `.gitignore:6-7`; `backend/.env` esta rastreado por Git.

**Evidencia:** el comentario declara que el archivo con credenciales se versiona. Contiene configuracion JWT, base de datos y bootstrap de administrador.

**Impacto:** un clon o historial puede exponer secretos de desarrollo; si se reutilizan fuera de local, habilitan acceso a base de datos, sesiones o administracion.

**Correccion:** sacar el archivo del indice, ignorarlo, conservar `.env.example` sin valores reales y rotar todos los secretos publicados.

**Ubicacion:** `backend/src/main.rs:89`.

**Evidencia:** `Cors::permissive()`. Una preflight desde `https://example.invalid` recibio ese origen en allow-origin y `access-control-allow-credentials: true`.

**Impacto:** peligroso para endpoints presentes o futuros autenticados por cookie; no debe ser el default de produccion.

**Correccion:** allowlist por entorno de origenes, metodos y cabeceras; desactivar credenciales si no se usan cookies.

### SEC-03 · Media · JWT de siete dias persistido en localStorage

**Ubicacion:** `frontend/src/utils/session.ts:8-31`, `backend/src/auth/services.rs:25-44` y `frontend/src/utils/api/client.ts:25-28`.

**Evidencia:** se guarda la sesion completa, incluido bearer token, en localStorage y se usa durante siete dias.

**Impacto:** XSS, extension maliciosa o equipo compartido pueden extraer una sesion reutilizable.

**Correccion:** cookie HttpOnly/Secure/SameSite con CSRF y rotacion; como transicion, token corto solo en memoria y revocacion.

### SEC-04 · Media · Sin limite visible de intentos de autenticacion

**Ubicacion:** `backend/src/auth/handlers.rs:20-25`, `backend/src/auth/services.rs:53-84`, `backend/src/main.rs:84-118`.

**Evidencia:** login y registro no pasan por rate limit; solo hay tracing, logging y CORS.

**Impacto:** facilita fuerza bruta, enumeracion y abuso de registro.

**Correccion:** rate limit por IP e identidad, backoff, limites de registro y telemetria de fallos.

### SEC-05 · Media · Sanitizador HTML por subcadenas

**Ubicacion:** `backend/src/internet/sanitize.rs:67-74`, `SiteFrame.tsx:113-121` y `siteRuntime.ts:121-129`.

**Evidencia:** bloquea `src=` y `url(` literal, pero HTML/CSS admite `src =` y `url (...)`. Luego sirve HTML por srcDoc y repite plantillas con innerHTML.

**Impacto:** puede romper la regla de no cargar recursos externos y habilitar seguimiento. El sandbox limita alcance, pero no convierte un filtro textual en parser seguro.

**Correccion:** sanitizador basado en parser y allowlist, politica CSS separada y pruebas de bypass.

### SEC-06 · Baja · Cabeceras defensivas no visibles

**Ubicacion:** `frontend/index.html:1-11` y respuesta local Vite.

**Evidencia:** no se ve CSP, frame-ancestors, nosniff, Referrer-Policy ni Permissions-Policy.

**Correccion:** configurarlas y verificarlas en proxy/CDN. Empezar CSP report-only y evitar unsafe-inline/unsafe-eval.

## Rendimiento y escalabilidad

### PERF-01 · Media · Bundle inicial unico

**Evidencia:** 441.02 kB de JS y 121.91 kB CSS; `frontend/src/main.tsx:6-23` importa App y todas las hojas de estilo de forma estatica.

**Recomendacion:** dividir rutas y panel admin con React.lazy, cargar estilos por area y definir presupuesto de bundle. Mantener login fuera de modulos pesados.

### PERF-02 · Media a futuro · El panel descarga todos los sitios

**Ubicacion:** `backend/src/internet/services.rs:699-716`.

**Recomendacion:** listado de metadatos y contenido bajo demanda; paginacion, filtro y cache. Aceptable para decenas, no para miles.

### PERF-03 · Baja · Consulta de rol por solicitud

**Ubicacion:** `backend/src/auth/middleware.rs:79-93`. La revocacion inmediata es una buena razon; medir p95 antes de cambiar. Si hace falta, usar cache breve o auth_version invalidable.

## Calidad de codigo y buenas practicas

### CODE-01 · Media · Puerta de calidad Rust no limpia

rustfmt detecta diferencias en muchos archivos. Clippy estricto falla por import sin uso, variante/estructuras/campos sin uso y una funcion sin uso. Aplicar rustfmt, retirar o justificar codigo muerto y exigir fmt, Clippy y tests en CI.

### CODE-02 · Media · Sin lint ni pruebas frontend automatizadas

**Ubicacion:** `frontend/package.json`. Solo define dev, build y preview. Anadir ESLint TypeScript/React, tests del cliente API y flujos Playwright de login/rutas criticas.

### CODE-03 · Baja · Modulos extensos

`internet.css` tiene 1570 lineas, `admin.css` 1107, `hardware.css` 980 y `utils/api/types.ts` 577. Partir por componente o dominio para reducir conflictos y mejorar navegacion.

> **Buenas practicas presentes.**
>
> - Consultas revisadas usan .bind, reduciendo SQL injection.
> - Contrasenas con bcrypt coste 13.
> - Autorizacion admin en servidor, no solo en UI.
> - Iframe sin allow-same-origin y validacion de event.source.
> - Lockfile y npm audit limpio para produccion.

## Estetica, UX y accesibilidad

### UX-01 · Alta · Colision de controles a 390 px

**Evidencia:** `output/playwright/login-normal-mobile.png` muestra titulo de ventana y selector Normal/Console superpuestos; tambien sucede en consola.

**Correccion:** bajo 480 px, permitir wrap, abreviar/ocultar titulo o llevar selector a fila propia. Convertir la captura movil en prueba de regresion.

### UX-02 · Media · Consola movil con scroll interno dificil de descubrir

**Evidencia:** `output/playwright/login-mobile.png` muestra contenido comprimido y scrollbar fino. Adaptar neofetch a una columna, ocultar datos secundarios o dejar crecer la ventana.

### UX-03 · Baja · Idioma y acabado de entrada

**Ubicacion:** `frontend/index.html:2`; favicon local devuelve 404. Usar `lang=es` o localizacion coherente y anadir favicon/manifest.

**Valoracion visual:** en escritorio, la consola tiene identidad, jerarquia y paleta consistentes; el problema principal es responsive, no direccion artistica.

## Plan de remediacion

1. Rotar secretos y sacar .env del historial operativo; configurar variables por entorno.
2. Reemplazar CORS permisivo por allowlist y anadir cabeceras de produccion.
3. Disenar sesion segura, rate limit y telemetria de autenticacion.
4. Sustituir filtro HTML por sanitizacion estructurada y pruebas de bypass.
5. Corregir movil, favicon e idioma; automatizar capturas a 390 px.
6. Limpiar rustfmt/Clippy, introducir lint/tests frontend y presupuesto de bundle.
7. Paginar sitios y medir autorizacion antes de optimizar.